

CSCE 633 MACHINE LEARNING · MODULE 08 · LECTURE NOTES

Clustering and Density

Finding structure when nobody labelled anything.

MODULE OUTCOMES

- Implement k-means and state its assumptions.
- Explain EM and Gaussian mixtures as soft clustering.
- Apply density-based clustering and say when it wins.
- Explain why cluster validation is fundamentally harder.
- Decide whether clustering is the right question.

1 k-means

```
initialise k centroids      (k-means++, not uniformly at random)
repeat until assignments stop changing:
  ASSIGN  each point to its nearest centroid
  UPDATE  each centroid to the mean of its assigned points
```

Coordinate descent on $\sum ||x - \mu_c||^2$: each step cannot increase the objective, so it CONVERGES -- to a LOCAL minimum that depends entirely on the initialisation.

WHAT IT QUIETLY ASSUMES

- * clusters are SPHERICAL (Euclidean distance)
- * clusters are SIMILAR SIZE (the mean is pulled by counts)
- * clusters have SIMILAR SPREAD (no per-cluster covariance)
- * k is known (it is not)
- * every point belongs somewhere (no concept of an outlier)

k-means always succeeds. Given uniform random noise and $k = 3$, it returns three clusters, with centroids and assignments and a plausible-looking plot. **It has no way to report that there was no structure**, which makes it unusually easy to produce a confident result from nothing — and the output is visually convincing, because any partition of a cloud looks like clusters when coloured.

INITIALISATION AND K ARE BOTH REAL PROBLEMS

Random initialisation produces bad local optima regularly, especially when two centroids start near each other and split a true cluster while merging two others. **k-means++** seeds centroids one at a time with probability proportional to squared distance from the nearest existing centroid, which spreads them out and is strictly better for negligible cost. **Use it; it is the library default for good reason.**

And run the whole thing several times, keeping the best objective. Ten restarts is cheap and standard, and the spread across restarts tells you how stable the solution is — which is itself informative.

Choosing k has no principled answer. The elbow method asks you to find a kink in a curve that is usually smooth, and different people reading the same plot pick different values.

The silhouette score and the gap statistic are better and are still heuristics (§3). If the value of k materially changes your conclusion, the conclusion is fragile — which is worth establishing before building anything on top of it, and is a good reason to check whether the downstream task is sensitive to k at all.

2 Mixtures and EM

A GAUSSIAN MIXTURE IS K-MEANS WITH UNCERTAINTY AND SHAPE

Model the data as generated by k Gaussian components, each with its own mean, covariance matrix, and mixing weight. This is a generative story: it says how the data could have been produced, which k-means does not.

EM alternates two steps. The E-step computes, for each point, its posterior *probability* of having come from each component. The M-step re-estimates each component's parameters as a weighted average using those probabilities.

So assignment is soft — a point can be 60% in one component and 40% in another, which is both more honest and directly useful — **and the per-component covariance lets clusters be elongated, rotated, and differently sized**, fixing two of §1's assumptions outright.

k-means is the limiting case with spherical equal covariances and hard assignment, which is worth seeing: the two algorithms are the same algorithm under different constraints. **EM also converges only to a local optimum**, needs k just as badly, and has an additional failure mode — **a component can collapse onto a single point, driving its covariance to zero and the likelihood to infinity**, which requires a regularisation floor on the covariance to prevent.

Method	What it finds	What it needs, and where it fails
k-means	Spherical, similar-sized groups.	Needs k. No outlier concept. Fails on elongated or nested shapes. Fast and the sensible first try.
Gaussian mixture / EM	Elliptical groups with soft membership.	Needs k, has many more parameters to estimate, and components can collapse. Gives probabilities, which is frequently the point.

Method	What it finds	What it needs, and where it fails
DBSCAN	Arbitrary shapes, and it labels points as noise.	Needs ϵ and a minimum point count, and fails when clusters have genuinely different densities — one ϵ cannot serve both.
HDBSCAN	Arbitrary shapes at varying density, with a cluster hierarchy.	Largely supersedes DBSCAN ; needs only a minimum cluster size, which is a far more intuitive parameter.
Hierarchical (agglomerative)	An entire tree of nested groupings, cut wherever you choose.	$O(n^2)$ memory, and the linkage choice — single, complete, average, Ward — changes the result substantially.
Spectral	Groups by connectivity in a similarity graph.	Elegant and handles shapes well ; the eigendecomposition is $O(n^3)$, so it does not scale.

DBSCAN's ability to declare a point noise is its single most useful property, and no centroid-based method has it — k-means must assign every point somewhere, so outliers drag centroids and corrupt the clusters they are nearest to.

3 Validation

THERE IS NO HELD-OUT SET FOR CLUSTERING

Supervised learning has ground truth, so a held-out score answers the question 'is this right?' directly. Clustering has no ground truth by construction — if you had the groups, you would not need to find them.

Internal measures — silhouette, Davies–Bouldin, Calinski–Harabasz — score compactness and separation. But that is a *definition* of a good clustering, not a measurement of one, and each measure favours the shape it implicitly assumes: silhouette rewards spherical compact clusters, so it will prefer k-means' answer to DBSCAN's on data where DBSCAN is right.

External measures — adjusted Rand index, normalised mutual information — compare against known labels. They are the honest option when labels exist, and if labels exist you should probably be doing supervised learning.

So the genuinely honest validation is downstream: does using these clusters improve some task you can actually measure? Customer segments that improve a campaign's response rate are validated; segments that look tidy are not. **Anything else is a preference presented as a result,** and recognising that is most of what this module is for.

CLUSTERING TELLS YOU ABOUT YOUR DISTANCE FUNCTION

Every clustering is relative to a notion of similarity, and you chose that notion — usually implicitly, by selecting features and deciding whether to scale them.

So change the scaling and the clusters change. With unscaled features, the one with the largest numeric range dominates the Euclidean distance almost entirely, and the 'discovered' clusters are bins of that single variable. This is not a subtle effect.

'The data has three natural clusters' is almost never a fact about the data. It is a fact about the data, plus your feature choice, plus your scaling, plus your metric, plus your k, plus your algorithm — six decisions, of which perhaps one was examined.

Which is why clustering results must be reported together with the full preprocessing, and why they transfer so badly between papers and between teams. **Treating a clustering as a discovery about the world rather than a summary under an assumed metric is the characteristic error of this area.**

4 Whether to cluster at all

- **Exploration.** Looking at the data before deciding what to model — **the strongest and most defensible use,** because the output is a hypothesis to check rather than a conclusion.
- **Compression and vector quantisation.** Replace points by their centroid and measure the reconstruction error. **This has a real objective and can be optimised honestly,** which makes it the one use where k-means' objective is the thing you care about.

- **Feature construction.** Cluster membership, or distance to each centroid, as a feature for a supervised model — **which can be evaluated**, on the downstream task, with a held-out set. This converts the unanswerable question into an answerable one.
- **Anomaly detection.** Density-based methods where 'belongs to no cluster' is exactly the output wanted, and the evaluation is against known anomalies.
- **And not for 'discovering natural categories'**, which presumes both that such categories exist and that your chosen metric reveals them — neither of which the method can check. **If a downstream task exists, evaluate on it**; that single move resolves most of §3's difficulty.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Hastie et al. — The Elements of Statistical Learning, chapter 14 (free PDF)	Clustering, EM, and the validation difficulty of §3, treated with appropriate scepticism.
Arthur & Vassilvitskii — k-means++: The Advantages of Careful Seeding (free)	The §1 initialisation, with the approximation guarantee.
Campello, Moulavi & Sander — HDBSCAN (free), and the documentation	The §2 method that supersedes DBSCAN , with unusually good documentation about when clustering is and is not appropriate.
von Luxburg, Williamson & Guyon — Clustering: Science or Art? (free)	The §3 and §4 argument, made properly — why clustering resists objective evaluation and what follows.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement k-means from scratch and verify the objective decreases monotonically.
- 2 **Run it on uniform random noise with $k = 4$** and plot the result. Note how convincing it looks.
- 3 Compare random initialisation against k-means++ over fifty runs, and report the distribution of final objectives.
- 4 **Construct data that breaks each of k-means' five assumptions** and show the failure in each case.
- 5 Implement EM for a Gaussian mixture and show it handles elongated clusters that k-means cannot.
- 6 **Make a component collapse** onto a single point and then prevent it with a covariance floor.
- 7 Run DBSCAN and HDBSCAN on nested-ring data and compare against k-means.
- 8 **Cluster the same data with and without feature scaling** and report how different the groupings are.

- 9 Compute silhouette for several k and for several algorithms. **Show that it prefers the algorithm whose assumptions it shares.**
- 10 Use **cluster membership as a feature** in a supervised model and evaluate on the downstream task.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe k-means and say what it converges to.
- 2 Name its five assumptions and why each matters.
- 3 Why is it dangerous that k-means always returns k clusters?
- 4 How does a Gaussian mixture generalise k-means, and what can go wrong?
- 5 Compare six clustering methods on what they find and need.
- 6 Why is clustering validation fundamentally harder than supervised evaluation?
- 7 What do internal measures actually measure?
- 8 Why is a clustering a fact about your metric?
- 9 Give four legitimate uses of clustering and one illegitimate one.